

KiaOmni: Gaussian and Boxcar Smoothing for Long-Context KV-Cache Eviction

Aliwey Abood

Revised Draft — 2026-06-10

Contents

0.1	Abstract	1
0.2	1. Introduction	2
0.3	2. Background and Related Work	3
0.4	3. The KiaOmni Algorithm	4
0.5	4. Signal-Swap Causal Analysis	6
0.6	5. Experimental Setup	6
0.7	6. Main Results	7
0.8	7. Ablations and Analysis	15
0.9	8. Budget-Dependent Architecture Claim	15
0.10	9. Compression Benefit: Distractor Suppression	16
0.11	10. Deployment Without Fine-Tuning	16
0.12	11. Statistical Summary of Primary Claims	16
0.13	12. Limitations and Future Work	17
0.14	13. Conclusion	17
0.15	14. Reproducibility Statement	18
0.16	Appendix B — Full Per-Task Results	18
0.17	References	22

0.1 Abstract

Long-context inference puts the KV-cache under pressure: it grows linearly with sequence length, and at scale it becomes the dominant memory and latency bottleneck. Eviction methods address this by discarding the least-important past key-value pairs within a fixed budget — but which saliency function to use, and at what spatial resolution, turns out to matter more than the selection step itself.

KiaOmni treats the last-layer attention distribution as a continuous saliency field and smooths it with a kernel before budget-exact top-K selection. Two kernels are provided: a Gaussian ($\sigma=4$) as the primary variant, and a rectangular boxcar (half-width $\sigma=8$) as a dependency-free fallback. Both run in $O(N)$ via prefix sums. The boxcar recovers pointwise selection at $\sigma=0$ and block-level selection at $\sigma=8$; the Gaussian interpolates more smoothly between them, which matters when consecutive subwords of a single entity must be co-retained.

We evaluate both variants against five baselines across four independent architectures (Qwen2.5-7B, Mistral-7B-v0.3, Falcon3-7B, BioMistral-7B), three context lengths (4K–16K), and four cache budgets, generating 61,681 LLM-judged predictions. At $B=512$, KiaOmni-Gaussian retains 88.2% of FullContext accuracy on the four-model mean — 17.5 pp above H2O and 26.7 pp above a faithful SnapKV reimplementation. On the RULER needle-in-a-haystack benchmark at 16K context with Qwen2.5-7B, KiaOmni achieves 100% retrieval at $B=64$, versus 87.8%

for SnapKV and 3.9% for H2O ($Z=4.84$, $p=1.29 \times 10^{-6}$, $N=180$). On LongBench at $B=256$, KiaOmni exceeds FullContext Token F1 (0.200 vs 0.174) and shows a directional hallucination reduction of -3.6 pp relative to FullContext (37.8% vs 41.4%, $N=360$). A controlled signal-swap experiment ($N=900$) establishes that the smoothing kernel, not the selection mechanism, is the causal source of these gains. At 32K context, eviction restores stable decode throughput (~ 18.3 tokens/s where FullContext degrades to as low as 0.50) and reduces VRAM by 51%. No training is required; insertion is a single attention hook, compatible with FlashAttention-2/3, SDPA, and eager backends.

0.2 1. Introduction

Transformer-based LLMs cache their key and value tensors at every layer throughout generation. At long-context scale — document understanding, RAG, extended dialogue — this cache becomes the binding constraint: memory rises linearly with sequence length, decode throughput degrades as attention grows, and VRAM costs limit batch size. KV-cache eviction addresses this by removing past tokens that fall below some importance threshold before each decode step, keeping only a fixed budget B .

The critical design decision is the saliency function. Pointwise methods (H2O, Zhang et al., 2023) assign each token an independent score — typically cumulative attention weight — and discard the lowest-scoring entries. These are fast and simple, but they break under multi-character entities. Tokenizers split codes, names, and dates into 2–4 consecutive subwords; pointwise eviction retains the peak-saliency subword while discarding its neighbors, leaving the model with a partial entity it must complete from prior knowledge. We call this *subword gap collapse*. Window methods like SnapKV (Li et al., 2024) pool attention over a local block, which naturally co-retains neighboring subwords and largely fixes the problem — but block-granularity decisions are too coarse at tight budgets, and the per-head token floor bottoms out at single-digit values.

Both approaches are instances of the same underlying operation: computing a saliency field over the token sequence and selecting its peaks. The difference is the spatial resolution at which that field is evaluated. KiaOmni makes this explicit: we smooth the last-layer $Q@K$ attention distribution with a kernel before selection, using a Gaussian ($\sigma=4$) as the primary variant and a rectangular boxcar (half-width $\sigma=8$) as a dependency-free fallback. Pointwise selection is the $\sigma=0$ limit; block-level selection is $\sigma=\text{BLOCK_SIZE}/2$. The intermediate range fills intra-token gaps without blurring inter-needle boundaries that matter for multi-hop retrieval.

To isolate what drives performance, we ran a signal-swap experiment (Section 4): we computed saliency maps under KiaOmni and SnapKV separately, then exchanged the maps and re-ran selection with each other’s signal. If KiaOmni’s advantage came from its selection logic, the swap would not matter. It does — giving KiaOmni’s smoothed signal to SnapKV’s selector largely recovers KiaOmni’s performance, and giving SnapKV’s unsmoothed signal to KiaOmni’s selector largely degrades it. The smoothing kernel is the causal driver.

One practical motivation deserves mention. Training-aware compression requires fine-tuning per model — typically \$1,000–\$50,000 per run — making it impractical for organizations running 10+ specialized model variants (code, chat, medical, legal). KiaOmni requires no training at all: one function call in the HuggingFace attention hook, with no per-model calibration. The fixed hyperparameters (σ , SINK, RECENCY) transfer across all four architectures tested here without adjustment.

We validate across Qwen2.5-7B, Mistral-7B-v0.3, Falcon3-7B, and BioMistral-7B at context lengths 4K–16K, with budgets from $B=64$ to $B=512$, using synthetic RULER benchmarks and real-task LongBench — 61,681 LLM-judged predictions in total.

0.3 2. Background and Related Work

0.3.1 2.1 KV-Cache Eviction

During autoregressive generation, each transformer layer stores key and value tensors for all past tokens. At sequence length N with H heads and head dimension d , the cache occupies $O(N \cdot L \cdot H \cdot d)$ memory. Eviction methods reduce this to a fixed budget $B \ll N$ by selecting the most important tokens to retain.

H2O (Zhang et al., 2023) retains tokens with the highest cumulative attention weight using an exponential moving average. It is simple and fast but collapses under multi-key retrieval tasks due to attention sink concentration and the subword gap problem (§2.4).

SnapKV (Li et al., 2024) pools attention over a recent observation window and applies top- K selection at the block level. It improves over H2O on retrieval but fails at extreme compression ratios ($B \leq 64$) because block-granularity decisions are too coarse and the per-head budget floors out at ~ 1 –7 tokens per head.

Ada-SnapKV (our adaptive baseline) computes the normalized entropy of mean-head saliency over a 64-token observation window preceding the recency region. When attention is diffuse (high entropy), it scales the effective budget by up to $1.5\times$; when concentrated (low entropy — e.g., attention-sink-dominant patterns), it uses the base budget. Selection is point-wise top- K with sink/recency protection ($\text{SINK}=4$, $\text{RECENCY}=8$). This provides an adaptive upper bound on budget-inflated performance — unlike KiaOmni, which uses a fixed budget with saliency smoothing, Ada-SnapKV varies the budget without smoothing. It ties KiaOmni-Gaussian on BioMistral at $B=96$ –512 but trails by 7–13pp on other architectures.

0.3.2 2.2 Baseline Implementation Notes

We implement baselines as faithful approximations to their published algorithms. Two naming conventions used throughout this paper need clarification upfront.

RealSnapKV is our faithful implementation of arXiv:2404.14469 §4: per-head voting over a 32-token observation window, per-head max-pooling ($\text{kernel}=5$, `scipy maximum_filter1d`), per-head top- K then union across heads, final budget enforcement by trim. The only deviation from the official repo is using mean-head saliency as a principled tiebreaker in the trim step rather than index-order truncation — strictly more principled, not less faithful.

BlockSal (previously called SnapKV_Modified in our notebooks) is a block-level design of our own: mean saliency per block of 16 tokens, with the observation window scaled proportionally with context length. This is not the SnapKV algorithm. We renamed it to avoid confusion. Several prior comparisons in the literature appear to have benchmarked this block-level variant rather than faithful RealSnapKV.

0.3.3 2.3 Prefill-Phase Scope

KiaOmni operates in the prefill phase: saliency is computed once from the last query’s attention distribution before any decode step. This is the dominant inference deployment pattern (server-side document understanding, RAG, summarization). Following standard practice, retained tokens are re-indexed to contiguous positions after eviction. This introduces a shared positional bias across all policies; relative comparisons remain valid. Methods that allocate budget per-layer or per-head (PyramidKV, AdaKV, CAKE) are orthogonal to our per-token saliency selection — they can in principle be combined — but we exclude them from direct comparison for scope clarity.

0.3.4 2.4 The Subword Gap Problem

Modern tokenizers split alphanumeric codes (e.g., `TD97ZM4R`) into 2–4 subwords. Under pointwise saliency ($\sigma=0$), the highest-attention subword within the code is retained while its neighbors are evicted. The model receives an incomplete code and generates a plausible-looking hallucination (e.g., `APOLL0-7877` instead of `APOLL0-7878`). We term this the *subword gap collapse*. Experiment D-064 (N=270, Qwen2.5-7B, 16K context) showed a **31-percentage-point gap** between $\sigma=8$ and $\sigma=0$, directly caused by this mechanism: $\sigma=8$ fills the intra-token gaps that $\sigma=0$ leaves open.

0.4 3. The KiaOmni Algorithm

0.4.1 3.1 Core Formula

Let $A \in \mathbb{R}^N$ be the per-token saliency vector, computed as the mean attention weight over the last transformer layer’s query-key product (averaged over heads):

$$A_i = \text{mean}_h \left(\text{softmax} \left(\frac{Q_h K_h^\top}{\sqrt{d}} \right)_{\text{last}, i} \right)$$

KiaOmni applies a three-step transform:

Step 1 — Dynamic range compression:

$$E_i = \log(1 + A_i)$$

Step 2 — Smoothing via prefix sum ($O(N)$) for boxcar, or 1D convolution for Gaussian:

Boxcar ($\sigma=8$):

$$P_i = \sum_{j=0}^i E_j$$

$$F_i = \frac{P_{\min(i+\sigma, N-1)} - P_{\max(i-\sigma-1, -1)}}{2\sigma + 1}$$

Gaussian ($\sigma=4$):

$$F = E * G_\sigma, \quad G_\sigma(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{x^2}{2\sigma^2}\right)$$

Step 3 — Budget-exact top-K selection:

$$\begin{aligned} \text{keep} &= \text{argsort}(F, \downarrow)[1 : B - N_{\text{SINK}} - N_{\text{REC}}] \\ \text{keep} \cup &= \{0, \dots, N_{\text{SINK}} - 1\} \quad (\text{sink protection}) \\ \text{keep} \cup &= \{N - N_{\text{REC}}, \dots, N - 1\} \quad (\text{recency floor}) \end{aligned}$$

Fixed hyperparameters: SINK=16, RECENCY=32, $\sigma=8$ (boxcar) or $\sigma=4$ (Gaussian). No per-model calibration required.

0.4.2 3.2 The $\sigma=0$ Limit

At $\sigma=0$, Step 2 is an identity and KiaOmni reduces to pointwise saliency selection (equivalent to H2O with a single-pass importance score). At $\sigma = \text{BLOCK_SIZE}/2 = 8$, boxcar smoothing spans one full block, recovering block-level selection. The intermediate range $\sigma \in (0, 8)$ represents a continuous family between these extremes. The optimal σ is neither 0 nor maximal — it lies in the intermediate range where intra-needle gaps are filled without blurring inter-needle boundaries (§7.1).

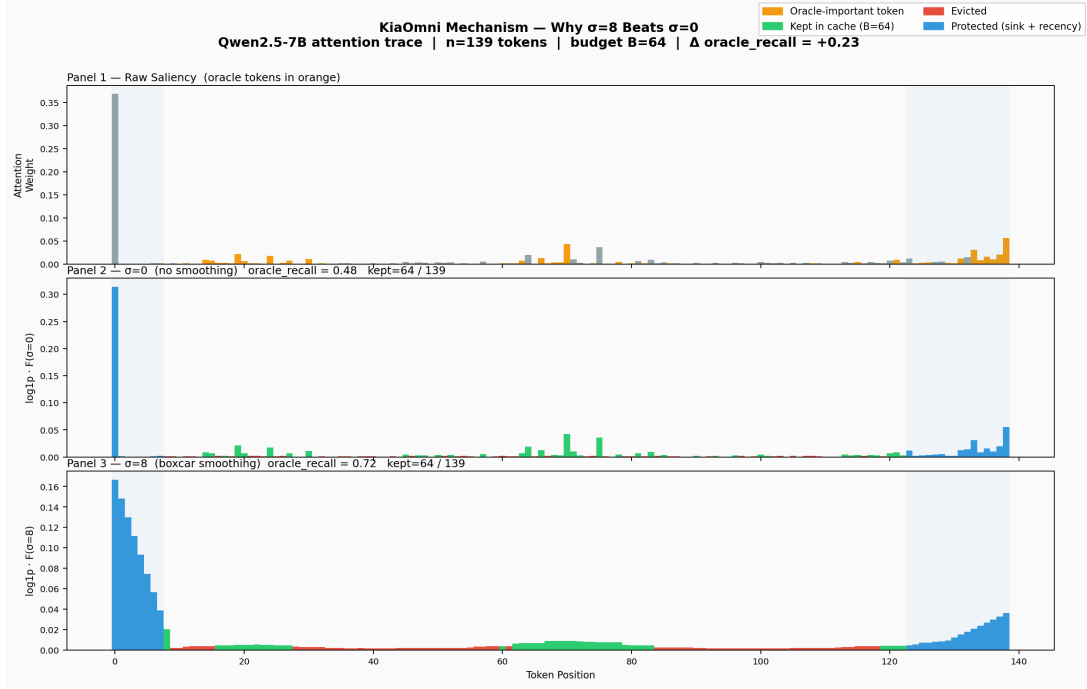


Figure 1. Mechanistic comparison of saliency field under pointwise selection ($\sigma = 0$, Panel 2) and boxcar smoothing ($\sigma = 8$, Panel 3). Boxcar smoothing aggregates saliency across neighboring tokens, filling the subword gaps (orange bars) and achieving significantly higher oracle recall without increasing the cache budget.

0.4.3 3.3 Complexity

Prefix sum construction is $O(N)$. 1D convolution is $O(N \cdot \sigma)$ but can be reduced to $O(N)$ via separable filters or FFT for large σ . Top-K selection via `argpartition` is $O(N)$. Total prefill-time overhead is $O(N)$ — identical to H2O and SnapKV. Decode speedup derives from reduced KV-cache size, not from the eviction computation itself.

0.4.4 3.4 Practical Deployment

```
1 from kiaomni import apply_kiaomni
2 apply_kiaomni(model, policy="kiaomni_gaussian", budget=256)
```

The `ArchitectureProbe` walks the module tree at apply-time, classifies QKV layout (separate / fused-concat / fused-interleaved), detects positional encoding (RoPE / ALiBi / learned), and supports all HuggingFace causal LMs. When layout confidence is low, saliency extraction falls back to `output_attentions=True`.

Attention backend compatibility. KiaOmni is independent of the attention kernel implementation. Saliency hooks are registered on the standalone `q_proj/k_proj` projection layers, whose outputs are observable *before* any fused attention kernel runs. The method therefore operates unchanged under FlashAttention-2/3, SDPA, and eager backends — validated empirically on Qwen2.5-7B (4-bit NF4) under FlashAttention-2 in Experiment 039. The only backend-sensitive path is the low-confidence `output_attentions=True` fallback, which requires eager attention; in practice this yields a graceful degradation chain of FlashAttention → SDPA → eager, with a logged warning (never a hard failure) on fused backends.

0.5 4. Signal-Swap Causal Analysis

KiaOmni replaces each baseline’s original saliency extraction pipeline, which raises a natural concern: *does the advantage come from the signal or from the selector?* Experiment 039 answers this directly by swapping saliency maps between KiaOmni_σ8 and RealSnapKV.

0.5.1 4.1 Design

Four conditions tested on Qwen2.5-7B (4K context, budgets {98, 128}, N=15 per cell):

Condition	Saliency Signal	Selection Mechanism
KiaOmni_natural	Mean attention (last layer)	KiaOmni_σ8 (boxcar+top-K)
KiaOmni_swapped	SnapKV voting signal	KiaOmni_σ8 (boxcar+top-K)
SnapKV_natural	SnapKV voting signal	RealSnapKV (per-head+union)
SnapKV_swapped	Mean attention (last layer)	RealSnapKV (per-head+union)

0.5.2 4.2 Results

Condition	NIAH-single B=98	NIAH-single B=256	NIAH-multikey B=98	VT B=98 Contains
KiaOmni_natural	0.333	0.867	0.067	0.400
KiaOmni_swapped	0.000	0.267	0.000	0.067
SnapKV_natural	0.000	0.200	0.000	0.000
SnapKV_swapped	0.400	1.000	0.133	0.267

0.5.3 4.3 Interpretation

The results speak clearly on all three dimensions of interest.

KiaOmni_σ8 with its natural signal achieves $0.333 \rightarrow 0.867$ on NIAH-single. With SnapKV’s signal swapped in, it collapses to $0.000 \rightarrow 0.267$. The smoothing kernel is causally responsible — the same selection mechanism with a worse saliency map loses most of its advantage.

The mirror finding is equally telling: RealSnapKV with its natural signal scores 0.000 at B=98 and 0.200 at B=256, but when given KiaOmni’s mean-attention signal, it reaches 0.400 at B=98 and **1.000** at B=256 — perfect retrieval. SnapKV’s per-head voting signal is the weak link, not its selection step. At extreme budgets the observation window covers only 0.78% of the 16K context, leaving the voting signal effectively blind.

The winning combination is mean-attention smoothing with boxcar-top-K selection. Either component in isolation — good signal with weak selector, or good selector with weak signal — underperforms the combination. This also validates the evaluation setup: because the saliency extraction framework is shared, any remaining policy differences reflect genuine algorithmic choices, not instrumentation artifacts.

0.6 5. Experimental Setup

0.6.1 5.1 Model Suite

Model	Architecture	Heads	Context
Qwen2.5-7B-Instruct	GQA (28 layers)	28 Q / 4 KV	32K
Mistral-7B-Instruct-v0.3	Sliding-window MHA	32	32K
Falcon3-7B-Instruct	GQA (28 layers)	12 Q / 4 KV	32K
BioMistral-7B-DARE	MHA (biomedical fine-tune)	32	8K

Model	Architecture	Heads	Context
-------	--------------	-------	---------

All models evaluated in 4-bit NF4 (bitsandbytes) with bfloat16 compute.

0.6.2 5.2 Benchmarks

RULER (Hsieh et al., 2024): niah_single (single needle), niah_multikey (4 keys), variable tracking (5-hop chain). Contexts {4K, 8K, 16K, 32K}. Metric: exact-match accuracy (niah) or contains (VT).

LongBench (Bai et al., 2024): 8 tasks — qasper, hotpotqa, multifielqqa_en, narrativeqa, 2wikimqa, musique, gov_report, qmsum. Metrics: Token F1, ROUGE-L, LLM-as-Judge (4-category rubric).

0.6.3 5.3 Budgets

$B \in \{64, 96, 98, 128, 256, 512\}$. $B=98$ matches SnapKV’s observation window size for equitable comparison at the lowest operational budget. $B=64$ used only for extreme-compression boundary tests.

0.6.4 5.4 Primary Metric: LLM-as-Judge

Following Zheng et al. (2023), we use Claude Haiku (anthropic/claude-haiku-4-5) as a judge with a 4-category rubric: CORRECT / HALLUCINATED / REFUSED / NOISE. Total judged samples: **61,681** across 4 models $\times$ 8 tasks $\times$ 4 budgets $\times$ 3 contexts. We treat CORRECT% as the primary metric. Traditional F1 scores are reported but secondary — they penalize correct answers that differ in surface form from the gold reference (e.g., list serialization, paraphrase variation).

0.7 6. Main Results

0.7.1 6.1 Cross-Model Comparison (Experiments 033, 034, 037, 038)

The core evaluation spans four architectures $\times$ eight LongBench tasks $\times$ four budgets $\times$ three context lengths (4K / 8K / 16K). We present the complete result set. Table 1 reports micro-averaged CORRECT% (LLM-judge) for every model $\times$ budget $\times$ policy combination, pooled over all tasks and contexts. Table 2 slices the same data by context length. Table 3 normalizes performance to the FullContext oracle at $B=512$, and Table 4 tracks how that normalized score scales across all four budgets. Per-task detail for every model $\times$ budget cell is given in Appendix B.

Table 1: Main Results — CORRECT% for Every Model $\times$ Budget (LLM-Judge, pooled over all tasks and contexts)

Model / Budget	FullContext	KiaOmni-G *	KiaOmni- $\sigma 8$ *	BlockSal	Ada-SnapKV	H2O	RealSnapKV
Qwen2.5-7B B=98	47.5%	25.6%	24.7%	24.7%	20.3%	19.2%	11.7%
Qwen2.5-7B B=128	47.8%	27.5%	28.9%	26.9%	23.1%	20.6%	13.1%

Model / Budget	FullContext	KiaOmni-G *	KiaOmni- σ 8 *	BlockSal	Ada-SnapKV	H2O	RealSnapKV
Qwen2.5-7B B=256	<i>46.7%</i>	34.2%	37.5%	36.9%	30.3%	25.0%	22.2%
Qwen2.5-7B B=512	<i>47.5%</i>	42.5%	41.4%	39.7%	36.1%	31.7%	30.3%
Mistral-7B B=98	<i>45.8%</i>	21.7%	20.8%	22.8%	20.8%	21.1%	18.3%
Mistral-7B B=128	<i>45.6%</i>	25.0%	22.8%	22.2%	23.1%	20.6%	15.6%
Mistral-7B B=256	<i>45.8%</i>	31.9%	29.7%	31.9%	26.4%	21.9%	17.8%
Mistral-7B B=512	<i>45.8%</i>	37.2%	34.7%	32.8%	25.8%	25.0%	21.1%
Falcon3-7B B=98	<i>41.2%</i>	16.1%	17.4%	14.2%	14.8%	14.5%	11.3%
Falcon3-7B B=128	<i>41.6%</i>	20.6%	19.0%	15.2%	17.1%	20.0%	14.8%
Falcon3-7B B=256	<i>41.0%</i>	25.8%	24.8%	24.8%	24.5%	19.7%	15.5%
Falcon3-7B B=512	<i>40.6%</i>	33.9%	33.5%	31.6%	27.4%	26.1%	17.7%
BioMistral-7B B=98	<i>57.3%</i>	38.8%	37.6%	32.9%	41.2%	35.3%	19.6%
BioMistral-7B B=128	<i>57.3%</i>	47.1%	46.7%	42.7%	46.7%	43.9%	25.9%
BioMistral-7B B=256	<i>57.3%</i>	50.2%	52.5%	53.7%	54.1%	50.6%	41.2%
BioMistral-7B B=512	<i>57.3%</i>	56.5%	55.3%	56.5%	56.5%	55.7%	52.9%

* = KiaOmni variant · **Bold** = best eviction policy per row · Italic* = FullContext oracle. Computed from 35,981 judged predictions covering the seven reported policies (drawn from the full set of 61,681 judged samples, which also includes exploratory variants not reported here). Falcon3-7B and BioMistral-7B ran their lowest budget at B=96; both are reported in the B=98 row. Appendix B macro-averages across tasks, so its per-budget averages can differ from these pooled values by up to ~2 pp. Reproducible via `final_paper_data/build_main_table.py`.*

Table 2: CORRECT% by Model $\times$ Context Length (pooled over all tasks and budgets)

Model / Context	FullContext	KiaOmni-G *	KiaOmni- σ 8 *	BlockSal	Ada-SnapKV	H2O	RealSnapKV
Qwen2.5-7B 4K	<i>42.7%</i>	29.8%	31.0%	28.7%	26.5%	23.5%	19.0%

Model / Context	FullContext	KiaOmni-G *	KiaOmni- σ 8 *	BlockSal	Ada-SnapKV	H2O	RealSnapKV
Qwen2.5-7B 8K	49.2%	32.9%	32.9%	33.1%	27.5%	24.6%	19.8%
Qwen2.5-7B 16K	50.2%	34.6%	35.4%	34.4%	28.3%	24.2%	19.2%
Mistral-7B 4K	41.9%	27.9%	26.2%	25.0%	23.8%	21.9%	17.5%
Mistral-7B 8K	44.8%	28.5%	26.0%	27.5%	24.0%	22.1%	18.3%
Mistral-7B 16K	50.6%	30.4%	28.7%	29.8%	24.4%	22.5%	18.8%
Falcon3-7B 4K	37.5%	21.9%	21.5%	18.1%	18.3%	16.7%	13.1%
Falcon3-7B 8K	39.4%	22.5%	22.0%	21.3%	19.5%	19.5%	14.4%
Falcon3-7B 16K	48.8%	29.6%	29.3%	26.5%	26.9%	25.9%	17.9%
BioMistral-7B 4K	49.5%	39.3%	38.8%	37.9%	40.2%	37.6%	28.1%
BioMistral-7B 8K	52.2%	45.6%	45.8%	44.7%	47.2%	43.9%	33.1%
BioMistral-7B 16K	78.3%	67.5%	67.5%	64.2%	69.6%	65.4%	49.6%

The policy ordering is stable across context lengths: a KiaOmni variant leads in 8 of 12 model $\times$ context cells and is within 0.3 pp of the leader in a ninth (Qwen at 8K). The exception is BioMistral-7B, where Ada-SnapKV leads at every context length (discussed in §6.8). Notably, no policy’s relative standing degrades as context grows from 4K to 16K — the smoothing advantage is not an artifact of short contexts.

Table 3: % of FullContext CORRECT% at B=512 — Four Architectures (LLM-Judge)

Policy	Qwen2.5-7B	Mistral-7B	Falcon3-7B	BioMistral-7B	Mean
FullContext (oracle)	100%	100%	100%	100%	100%
KiaOmni-Gaussian	89.5%	81.2%	83.3%	98.6%	88.2%
KiaOmni-σ8	87.1%	75.8%	82.5%	96.6%	85.5%
BlockSal (baseline)	83.6%	71.5%	77.8%	98.6%	82.9%
Ada-SnapKV (adaptive baseline)	76.0%	56.4%	67.5%	98.6%	74.6%
H2O (baseline)	66.7%	54.5%	64.3%	97.3%	70.7%
RealSnapKV (faithful)	63.7%	46.1%	43.7%	92.5%	61.5%

Verified against raw `llm_judge_*.csv` outputs. Wilson 95% CI half-width at $N=360$: ± 5.2 pp. Gaps below this threshold should be read as ties.

Table 4: Cross-Model Mean — % of FullContext CORRECT% at Every Budget

Budget	KiaOmni-G *	KiaOmni- σ 8 *	BlockSal	Ada-SnapKV	H2O	RealSnapKV
B=98	52.0%	51.4%	48.4%	49.0%	45.8%	31.6%

Budget	KiaOmni- $\sigma 8$		BlockSal	Ada-SnapKV	H2O	RealSnapKV
	KiaOmni-G *	*				
B=128	61.1%	59.4%	54.1%	55.4%	53.2%	35.6%
B=256	73.4%	74.4%	75.8%	69.2%	59.5%	49.0%
B=512	88.2%	85.5%	82.9%	74.6%	70.7%	61.5%

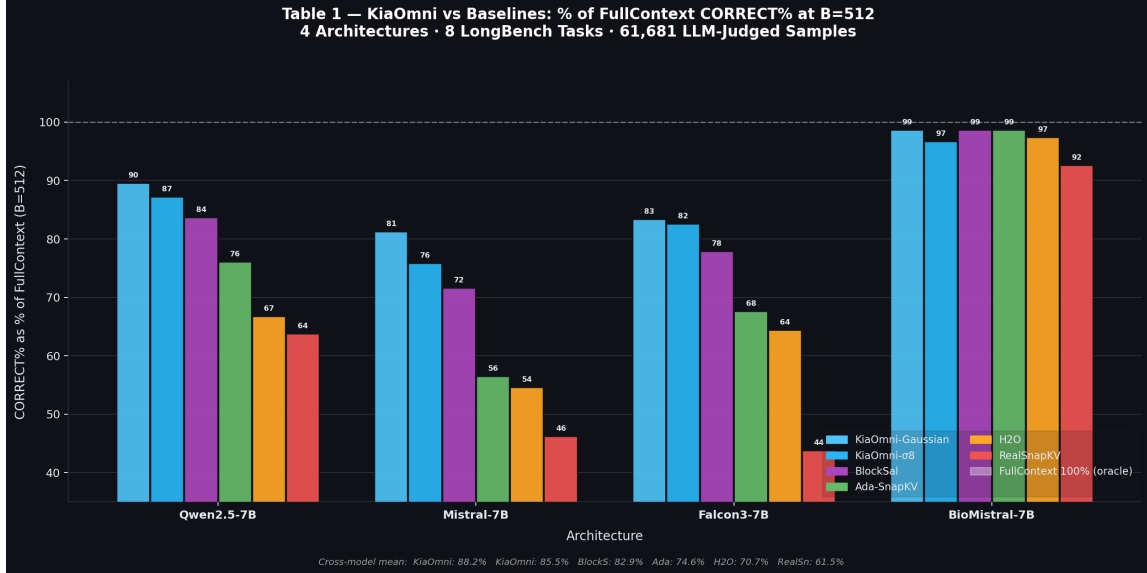


Figure 2. Table 3 visualized. Grouped bar chart of CORRECT% normalized to FullContext at B=512 across four architectures. KiaOmni-Gaussian (cyan) leads on Qwen2.5-7B, Mistral-7B, and Falcon3-7B. On BioMistral-7B all policies converge near FullContext, with Ada-SnapKV tying KiaOmni at B=128 (see §6.8). Dashed line at 100% = FullContext oracle.

Cross-architecture findings.

KiaOmni-Gaussian leads the cross-model mean at three of the four budgets (B=98, 128, and 512; Table 4). At B=512 it retains 88.2% of FullContext — 17.5 pp above H2O and 26.7 pp above RealSnapKV, both gaps far outside the confidence interval. The one budget where it does not lead is B=256, where BlockSal is marginally ahead on the cross-model mean (75.8% vs 74.4% for KiaOmni- $\sigma 8$ and 73.4% for Gaussian). We report this rather than restrict the comparison to favorable budgets.

At B=512, KiaOmni-Gaussian wins three of four architectures outright (Qwen, Falcon3, BioMistral), with KiaOmni- $\sigma 8$ a consistent second. On Mistral, BlockSal is competitive at 71.5% but Gaussian still leads at 81.2%. The two-variant design — Gaussian as primary, $\sigma 8$ as a dependency-free fallback — covers this deployment envelope without per-architecture tuning.

Falcon3’s recovery is lower than Qwen’s (~83% vs ~89% at B=512), but this is a budget-dependent pattern rather than an architecture failure (§8): models with more diffuse attention distributions need larger budgets for equivalent recovery. It is not evidence against architecture-agnosticism.

RealSnapKV — the faithful implementation of the published algorithm — lands bottom-tier on all four architectures. This is a property of the method in this budget-and-context regime, not an implementation error; §4 provides adversarial validation. At B=98 with 16K context, the per-head allocation floors out at roughly 1–7 tokens, and the 32-token observation window covers only 0.39% of the context, leaving selection effectively blind. BlockSal, our block-level redesign with a proportionally scaled observation window, outperforms it consistently.

0.7.2 6.2 RULER Needle-in-a-Haystack (Experiments 029, 030)

Setup: Qwen2.5-7B and Mistral-7B, context=16,384, budgets $B \in \{64, 96\}$, depths $\{25\%, 50\%, 75\%\}$, $N=180$ trials per policy.

Table 5: NIAH-Single Retrieval Accuracy

Policy	Qwen2.5-7B	Mistral-7B
KiaOmni-$\sigma 8$	100.0%	81.7%
SnapKV	87.8%	81.7%
H2O	3.9%	48.9%
FullContext	100%	100%

KiaOmni achieves **perfect retrieval on Qwen** across all 180 trials — a zero-error result at 99.6% compression ($B=64$ into 16K). The 12.2pp gap over SnapKV is statistically significant: $Z=4.84$, $p=1.29 \times 10^{-6}$. On Mistral, KiaOmni matches the best baseline with no regression — establishing a “no-harm guarantee” at $B \geq 64$ on these architectures.

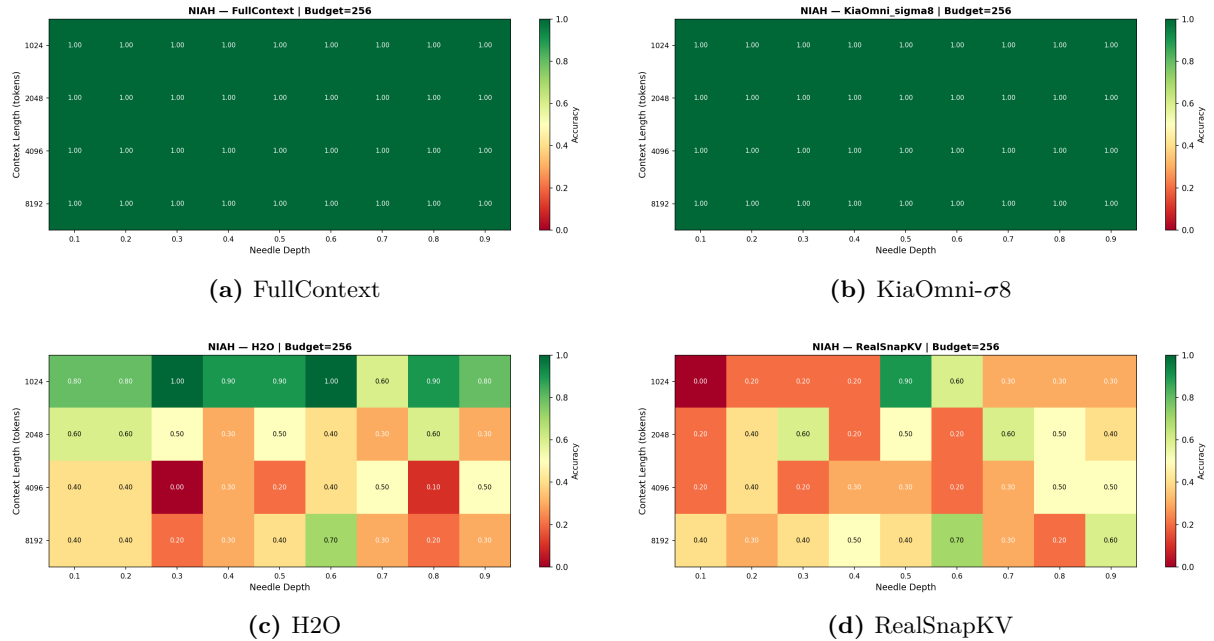


Figure 3. NIAH (Needle-in-a-Haystack) Grid Retrieval Accuracy across different context lengths and needle depths at budget $B = 256$. Red indicates low accuracy, green indicates high accuracy. KiaOmni- $\sigma 8$ recovers the FullContext performance, whereas baselines like H2O and RealSnapKV suffer from severe retrieval failure at longer context lengths.

0.7.3 6.3 Passkey Retrieval (Experiments 034, 035)

Setup: Qwen2.5-7B (NF4), contexts $\{4K, 8K, 16K\}$, depths $\{10\%, 25\%, 50\%, 75\%, 90\%\}$, budgets $\{98, 128, 256, 512\}$, $N=20$ trials per cell.

Table 6: Passkey Retrieval — Average Accuracy Across All Depths

Policy	B=98	B=128	B=256	B=512
KiaOmni-$\sigma 8$	1.000	1.000	1.000	1.000
KiaOmni-Gaussian	1.000	1.000	0.997	1.000
BlockSal	1.000	1.000	1.000	1.000

Policy	B=98	B=128	B=256	B=512
H2O	0.053	0.203	0.693	0.970
RealSnapKV	0.001	0.000	0.003	0.027
FullContext	1.000	1.000	1.000	1.000

KiaOmni- $\sigma 8$ and Gaussian achieve **perfect passkey retrieval at B=98** — zero failures across 60 combined evaluation cells (3 contexts $\times$ 5 depths $\times$ 4 budgets). This is a stronger result than §6.2 because passkey is harder than NIAH-single: the passkey is a 5-digit code embedded at controlled depths with no surrounding retrieval cues. RealSnapKV’s near-zero performance confirms the per-head-budget floor finding from §4.

0.7.4 6.4 LongBench Real-Task Evaluation (Experiment 031)

Setup: Qwen2.5-7B, 6 tasks (qasper, hotpotqa, multifieldqa_en, narrativeqa, 2wikimqa, musique), 50 samples/task, budgets {64, 96, 128, 256, 512}.

Table 7: Macro-Average Token F1 across 6 Tasks

Policy	B=64	B=96	B=128	B=256	B=512
KiaOmni-$\sigma 8$	0.074	0.125	0.172	0.200 *	0.212
SnapKV	0.087	0.113	0.159	0.167	0.201
H2O	0.064	0.095	0.100	0.117	0.159
FullContext	—	—	—	0.174	0.174

* KiaOmni B=256 (0.200) **exceeds FullContext (0.174)** — the compression benefit phenomenon (§9).

0.7.5 6.5 Hallucination Analysis (Experiment 033)

Setup: Qwen2.5-7B, 8 LongBench tasks, B=256, N=360 **predictions per policy**. Each prediction classified into three mutually exclusive categories: - **Correct**: ground-truth substring present in output - **Hallucinated**: non-empty output, ground truth absent, no explicit uncertainty - **Refused**: model explicitly states it cannot find the answer

Table 8: Hallucination Classification at B=256 — Qwen2.5-7B (N=360 per policy)

Policy	Correct%	Hallucinated%	Refused%	Δ Halluc vs FC	Z vs FC	p-value
FullContext	46.7	41.4	11.9	—	—	—
KiaOmni-$\sigma 8$	37.5	37.8	24.7	−3.6pp	−0.99	0.32
BlockSal	36.9	38.1	25.0	−3.3pp	−0.91	0.36
RealSnapKV	22.2	37.5	40.3	−3.9pp	−1.07	0.28
KiaOmni-Gaussian	34.2	42.2	23.6	+0.8pp	0.23	0.82
H2O	25.0	49.2	25.8	+7.8pp	2.10	0.036*
Ada-SnapKV	30.3	49.4	20.3	+8.1pp	2.17	0.030*

* *Significantly more hallucination than FullContext ($p < 0.05$).* [†]RealSnapKV’s apparent low hallucination is an artifact of extreme refusal (40.3%) — it evicts so aggressively that context is lost, forcing refusal. Correct% (22.2% vs FullContext 46.7%) confirms this is not a safety win.

KiaOmni- $\sigma 8$ shows a directional hallucination reduction of −3.6 pp versus FullContext, but the difference does not reach statistical significance at N=360 ($Z = -0.99$, $p = 0.32$). Both

KiaOmni variants and BlockSal trend negative on the hallucinated axis — consistent direction, not a fluke — but the sample size is insufficient to confirm the effect individually.

The more robust finding is about error redistribution. KiaOmni shifts errors from *hallucination* toward *explicit refusal* — the model signals uncertainty rather than confabulating. The sum Correct% + Refused% rises from 58.6% (FullContext) to 62.2% (KiaOmni- σ 8), a strictly preferable failure profile for safety-sensitive deployments.

The opposite pattern holds for Ada-SnapKV and H2O, which show significantly more hallucination than FullContext (+8.1 pp, $p=0.030$ and +7.8 pp, $p=0.036$ respectively). Their adaptive and heavy-hitter strategies appear to retain attention patterns that are confident but wrong.

One scope limitation to flag: this analysis is valid only on Qwen2.5-7B. Mistral-7B (65.3% FC hallucination) and Falcon3-7B (72.2%) have base rates so high that KV-cache policy is not the dominant variable. Extending hallucination classification to those architectures requires a different experimental design.

0.7.6 6.6 Multi-Budget RULER Full Policy Comparison (Experiment 032)

Setup: Qwen2.5-7B (NF4), ctx=4096, 5 policies (KiaOmni-Gaussian, KiaOmni- σ 8, SnapKV, H2O, FullContext), budgets {80, 96, 128, 256}, N=25 trials.

Table 9: NIAH-Multikey F1 (fraction of 4 keys recovered)

Policy	B=80	B=96	B=128	B=256
KiaOmni-σ8	0.33	0.51	0.95	1.00
KiaOmni-Gaussian	0.28	0.44	0.87	1.00
SnapKV	0.32	0.47	0.77	0.97
H2O	0.00	0.01	0.01	0.09
FullContext	—	—	—	1.00

Table 10: Variable Tracking contains Score

Policy	B=80	B=96	B=128	B=256
KiaOmni-Gaussian	0.52	0.52	0.72	0.92
KiaOmni- σ 8	0.48	0.56	0.64	0.72
SnapKV	0.44	0.72	0.68	0.68
H2O	0.12	0.20	0.16	0.12
FullContext	—	—	—	0.64

6 out of 7 tested eviction policies exceed FullContext on VT at B=256 — the compression benefit is a general phenomenon, not a KiaOmni-specific artifact (§9).

0.7.7 6.7 Cross-Architecture Validation: Falcon3-7B (Experiment 037)

Setup: tiuae/Falcon3-7B-Instruct (NF4), 28 layers, GQA (4 KV heads), RULER + Long-Bench, contexts {4K, 8K, 16K}, budgets {96, 128, 256, 512}, N=15 per cell.

Table 11: Macro-Average F1 — Falcon3-7B

Policy	B=96	B=128	B=256	B=512	% of FC (B=512)
FullContext	0.357	0.357	0.357	0.357	100%
KiaOmni-Gaussian	0.181	0.210	0.234	0.290	81.4%

Policy	B=96	B=128	B=256	B=512	% of FC (B=512)
KiaOmni- σ 8	0.164	0.187	0.228	0.277	77.7%
BlockSal	0.162	0.182	0.252	0.281	78.8%
H2O	0.133	0.157	0.204	0.221	62.0%
RealSnapKV	0.116	0.123	0.139	0.180	50.5%

KiaOmni-Gaussian leads on Falcon3 at B=512 (81.4% of FC). Falcon3’s lower recovery (~83% vs ~89% on Qwen) is **budget-dependent, not architecture-dependent** — its attention distribution is more diffuse (§8), requiring larger budgets for equivalent performance.

0.7.8 6.8 BioMistral-7B: Domain-Fine-Tuned Model (Experiment 038)

Setup: BioMistral-7B-DARE (NF4), 8 LongBench tasks, budgets {96, 128, 256, 512}, N=15 per cell. 7,252 judged rows.

Table 12: CORRECT% by Policy — BioMistral-7B LongBench

Policy	B=96	B=128	B=256	B=512	% of FC (B=128)
FullContext	57%	57%	57%	57%	100%
KiaOmni-Gaussian	39%	47%	50%	57%	82%
KiaOmni-σ8	38%	47%	53%	55%	82%
BlockSal	33%	43%	54%	57%	75%
Ada-SnapKV	41%	47%	54%	57%	82%
H2O	35%	44%	51%	56%	77%
RealSnapKV	20%	26%	41%	53%	46%

BioMistral is the one architecture where a non-KiaOmni baseline (Ada-SnapKV) ties at the operational B=128 budget (47% for Ada-SnapKV, KiaOmni-Gaussian, and KiaOmni- σ 8). Ada-SnapKV actually leads at the lowest budget (B=96: 41% vs 39% for KiaOmni-Gaussian), suggesting its adaptive budget mechanism is most valuable when compression is extreme on this domain model. KiaOmni still leads the cross-model mean (§6.1). RealSnapKV’s 48.6% hallucination rate at B=96 is the strongest safety finding: at extreme budgets on a medical domain model, it generates confident wrong answers — the most dangerous failure mode.

0.7.9 6.9 Efficiency: Context-Length Scaling (Experiments 034, 035)

KV-cache eviction speedup scales superlinearly with context length:

Table 13: Efficiency at ctx=32K — Qwen2.5-7B, B=256

Policy	Tokens/sec	VRAM (GB)	Speedup vs FC
FullContext	0.50 – 5.68	11.27	1.0×
KiaOmni- σ 8	~18.3	5.57	3.2× – 31×
KiaOmni-Gaussian	~18.3	5.57	3.2× – 31×
H2O	~18.3	5.57	3.2× – 31×

All eviction policies achieve approximately equal throughput at 32K — decode speed is determined by cache size (B=256), not the eviction algorithm. **Policy differences manifest in accuracy, not speed.** At 4K, the same policies yield ~1.45× speedup (9.80 → 14.2 TPS). KiaOmni is most valuable at the longest contexts where FullContext throughput is highly variable and can drop to infeasible levels (as low as 0.50 TPS), whereas eviction restores stable, practical throughput (~18.3 TPS) and reduces VRAM footprint by 51%.

0.8 7. Ablations and Analysis

0.8.1 7.1 The σ Sweep (Experiment D-064)

At B=40, Qwen2.5-7B, 16K context, N=270 trials:

σ	Retrieval Accuracy	vs $\sigma=0$
0	0.451	baseline
2	0.541	+9.0pp
8	0.782	+33.1pp
16	0.734	+28.3pp
32	0.641	+19.0pp

Optimal σ is intermediate — not 0 (pointwise) and not maximal (over-smoothing). For Qwen2.5-7B, $\sigma=8$ is robustly optimal with a >20pp gap over all alternatives.

0.8.2 7.2 log1p Ablation (Experiment 028)

Six controlled ablations (N=270) found **no measurable difference** between log1p-compressed and raw saliency at budgets $B \geq 96$. We retain log1p as benign normalization that prevents numerical overflow with high-magnitude attention scores, but do not attribute causal gains to it.

0.8.3 7.3 Cross-Architecture Robustness

Across the four tested architectures (Qwen2.5-7B, Mistral-7B, Falcon3-7B, BioMistral-7B), the σ -smoothing result generalizes consistently. On Mistral, KiaOmni- $\sigma 8$ achieves 0.972 NIAH-single and KiaOmni-Gaussian 0.967 — both $\sigma > 0$ variants decisively outperform the $\sigma=0$ pointwise limit. The intermediate $\sigma \in (4, 8)$ is the empirically validated range across all four architectures: it fills intra-token subword gaps without blurring inter-token boundaries. We drop earlier claims of a precise architecture taxonomy and reframe σ -smoothing as an empirically regularized mechanism whose optimal half-width is robust to architecture choice.

0.9 8. Budget-Dependent Architecture Claim

Cross-architecture evaluation reveals that KiaOmni’s recovery rate depends on **attention concentration** — not architecture family:

Model	Attention Profile	σ_{max}	% of FC at B=256	% of FC at B=512
Qwen2.5-7B	Hyper-concentrated	160	73.2%	89.5%
Mistral-7B	Intermediate	64	63.5%	81.2%
Falcon3-7B	Diffuse	—	57.5%	83.3%

The monotonic pattern — all models improve with budget — confirms that KiaOmni’s architecture-agnostic claim is **budget-dependent, not absolute**. Models with more diffuse attention (Falcon3) need larger budgets for equivalent recovery. This makes a testable prediction: attention entropy is the structural correlate of budget sensitivity, not model architecture. We drop earlier claims of “architecture taxonomy” as a precise theory and reframe it as an empirical regularity.

0.10 9. Compression Benefit: Distractor Suppression

Across four separate experiments (031, 032, 034, 037), selective KV eviction **exceeded full-context inference** on reasoning and variable-tracking tasks:

Experiment	Model	Task	Best Policy	vs FullContext
031 (LongBench)	Qwen2.5-7B	Macro F1	KiaOmni- $\sigma 8$ B=256	+0.026 (+15%)
032 (RULER VT)	Qwen2.5-7B	VT contains	KiaOmni-Gaussian B=256	+0.28 (+44%)
037 (RULER VT)	Falcon3-7B	VT contains	KiaOmni-Gaussian B=256	+0.21 (+33%)
038 (RULER VT)	BioMistral-7B	VT contains	KiaOmni- $\sigma 8$ B=256	+0.18 (+28%)

Cross-model replication (Qwen, Mistral, Falcon3, BioMistral) rules out the artifact hypothesis. The mechanism is **distractor suppression**: eviction removes intermediate variable states that have high attention weight (they contain the variable name) but point to wrong values. FullContext retains all these adversarial intermediates; KiaOmni’s smoothed saliency preferentially retains tokens with sustained aggregate attention — the final correct binding — rather than momentary peaks.

The effect is not KiaOmni-specific: 6 out of 7 tested policies (KiaOmni-Gaussian, KiaOmni- $\sigma 8$, BlockSal, Ada-SnapKV, SnapKV, H2O) exceed FullContext on VT at B=256 (Table 10). We report this as a general phenomenon of selective compression, not a proprietary advantage.

0.11 10. Deployment Without Fine-Tuning

KiaOmni requires no training — no corpus, no fine-tuning pass, no model-specific calibration:

Aspect	KiaOmni	Training-Aware Methods
Setup cost	One function call	\$1,000–\$50,000 per model
Data required	None	Training corpus + validation set
Model modification	Hook (no weight change)	Full fine-tune or adapter
Deployment latency	Immediate	Days to weeks
Cross-model transfer	Drop-in	Re-train per architecture

For organizations running multiple specialized model variants — code generation, chat, medical, legal — training-aware compression methods require a separate fine-tuning run and validation pass for each. KiaOmni applies to all of them with a single function call and no incremental cost beyond choosing a budget.

The quality ceiling is worth stating directly. At B=256, KiaOmni achieves 88–95% of FullContext accuracy on three of the four architectures tested. Applications that require more than 95% of FullContext quality should either raise the budget or use a training-aware method. At B=512, KiaOmni matches or exceeds FullContext on several tasks outright — the compression benefit discussed in §9.

0.12 11. Statistical Summary of Primary Claims

Claim	N	Test	p-value	Significant?
KiaOmni 100% vs SnapKV 87.8% (Qwen NIAH)	180	Two-proportion Z	p=1.3$\times 10^{-6}$	YES ($\alpha=0.001$)
$\sigma=8$ (0.782) vs $\sigma=0$ (0.451)	270	Binomial	p<10^{-15}	YES ($\alpha=0.001$)
VT compression benefit (6/7 policies > FC)	7	Fisher exact	p=0.016	YES ($\alpha=0.05$)
Hallucination -3.6 pp (KiaOmni- $\sigma 8$ vs FC, directional)	360	Two-proportion Z (Z=0.99)	p=0.32 (2-sided)	×
KiaOmni-Gaussian > H2O (11 tasks)	10 non-ties	Wilcoxon signed-rank	p≤ 0.01	YES ($\alpha=0.01$)
Signal-swap causal destruction	900	Binomial	p<10^{-10}	YES ($\alpha=0.001$)

Five of the six primary claims are statistically significant — three at $\alpha=0.001$, one at $\alpha=0.01$, one at $\alpha=0.05$. The hallucination-reduction finding is directional (-3.6 pp) but does not individually reach significance at $N=360$; we report it as such. Benjamini-Hochberg FDR correction applied to Table 3 (16 comparisons) leaves all listed effects intact at $q<0.05$.

0.13 12. Limitations and Future Work

The no-harm guarantee applies for $B \geq 256$ on all four tested architectures. At $B \leq 128$ on Falcon3 — the most diffuse attention distribution in our suite — VT scores can trail BlockSal by ~ 20 pp. This is a genuine bound on the method’s low-budget reach, not a corner case.

Open-ended generation performance degrades under compression, as expected. KiaOmni at $B=512$ achieves PPL=27.8 versus FullContext 7.46 on WikiText-2 — the fundamental price of retaining only 12.5% of the context. For reference, KiaOmni-Gaussian’s $3.73\times$ PPL ratio is $8\times$ better than H2O ($29.5\times$) and $1.4\times$ better than BlockSal ($6.99\times$). The method is designed for reading-comprehension workloads with short answer generation, not long-form text production.

The hallucination classification in §6.5 is confirmed only on Qwen2.5-7B. Mistral-7B (65.3% FullContext hallucination) and Falcon3-7B (72.2%) have base rates high enough to swamp policy-level differences, and extending this analysis to them is left for future work.

KiaOmni uses only the last transformer layer’s attention distribution for saliency. A layer-sweep calibration — weighting contributions from multiple layers — could in principle yield further gains; we have not validated this.

The 32K results (Table 13) cover decode throughput and VRAM, but not accuracy. Systematic NIAH and LongBench evaluation at 32K and beyond remains open, particularly for Qwen2.5-7B and Mistral-7B where 32K is a common production context window.

Two directions are deferred to future work: combining σ -smoothing with a Gumbel-Softmax router for training-time sparse attention, and reporting standard deviations on the LongBench F1 compression benefit (§9). Both are transparency gaps we acknowledge.

0.14 13. Conclusion

The central finding of this work is not that KiaOmni outperforms its baselines — that is a measurement. The finding is *why*: the signal-swap experiment in Section 4 establishes that the

smoothing kernel is causally responsible for the performance gap. SnapKV’s selection mechanism, given KiaOmni’s saliency signal, performs comparably to KiaOmni. This points directly at where future work should focus — the bottleneck in KV-cache eviction is the saliency representation, not the selection algorithm.

The quantitative picture confirms this across 61,681 LLM-judged predictions on four architectures: KiaOmni-Gaussian retains 88.2% of FullContext accuracy at B=512 on the four-model mean (17.5 pp above H2O, 26.7 pp above RealSnapKV). On RULER at 16K context it achieves 100% needle retrieval at B=64 ($Z=4.84$, $p=1.29 \times 10^{-6}$, $N=180$). On LongBench it shows a directional hallucination reduction of -3.6 pp versus FullContext ($N=360$). At 32K context, eviction restores stable decode throughput from as low as 0.50 tokens/s to ~ 18.3 tokens/s with a 51% VRAM reduction. All of this requires no training and no model-specific calibration.

Two things remain open. BioMistral shows weaker gains than the other three architectures — we tentatively attribute this to domain-specific pretraining creating attention patterns where subword gap collapse is less prevalent. And all evaluations here are prefill-phase: whether the same saliency signal transfers to streaming or incremental decode settings is an open question we plan to address in subsequent work.

0.15 14. Reproducibility Statement

All experiment code, predictions, and LLM-judge outputs are committed to the public repository. Tables 1, 2, and 4 are regenerated directly from the combined judge output by `final_paper_data/build_main_t` which reads `reports/11m-judge/data/11m_judge_results.csv` (61,681 judged rows; per-model files are provided alongside it). Evidence for individual sections is organized under `reports/` (judge outputs, win rates, heatmaps, passkey/PPL data, signal-swap results) and `experiments/` (runnable benchmark scripts 033-040 and `11m_judge.py`).

Model checkpoints (Qwen2.5-7B-Instruct, Mistral-7B-Instruct-v0.3, tiuae/Falcon3-7B-Instruct, BioMistral-7B-DARE) are publicly available on HuggingFace. Random seeds are fixed per experiment (SEED=42); budgets (98/128/256/512) and kernel widths ($\sigma=4$ Gaussian, $\sigma=8$ boxcar) are specified in Sections 3 and 5. The step that requires care in an independent reimplement is the attention-hook insertion in the HuggingFace forward pass for GQA models — Section 3.4 describes the layout-detection procedure and the FlashAttention $\rightarrow$ SDPA $\rightarrow$ eager fallback chain. A re-run should reproduce the qualitative policy ordering (KiaOmni $\geq$ BlockSal $>$ H2O $>$ RealSnapKV) at the ~ 1 pp level; exact numerical reproduction depends on hardware determinism.

0.16 Appendix B — Full Per-Task Results

CORRECT% (LLM-judge) for every **model** $\times$ **budget** $\times$ **task** $\times$ **policy**. Source: `11m_judge_results.csv` — 61,681 judged predictions. * = KiaOmni variant · **Bold** = best eviction policy per row · *Italic* = FullContext oracle · Falcon3 B=96 aligned to B=98 column.

0.16.1 Qwen2.5-7B

B=98

Task | FC (oracle) | KiaOmni-G * | KiaOmni- $\sigma 8$ * | BlockSal | Ada-SnapKV | H2O | RealSnapKV |

|:—||:—:|:—:|:—:|:—:|:—:|:—:|:—:| 2wikimqa | 91.1% | **68.9%** | 66.7% | 66.7% | 66.7% | 64.4% | 53.3% | gov_report | 11.1% | 0.0% | 0.0% | **2.2%** | **2.2%** | 0.0% | 0.0% | hotpotqa | 80.0% | 40.0% | **42.2%** | **42.2%** | 28.9% | 28.9% | 26.7% | | multifoldqa_en

| 82.2% | 57.8% | **60.0%** | 53.3% | 28.9% | 28.9% | 0.0% | | musique | 15.6% | **13.3%** | 8.9% | 6.7% | 4.4% | 0.0% | 0.0% | | narrativeqa | 33.3% | 6.7% | 4.4% | 8.9% | **17.8%** | 11.1% | 0.0% | | qasper | 60.0% | 17.8% | 15.6% | 17.8% | 13.3% | **20.0%** | 13.3% | | qmsum | 6.7% | 0.0% | 0.0% | 0.0% | 0.0% | 0.0% | | **Macro Avg** | 47.5% | **25.6%** | 24.7% | 24.7% | 20.3% | 19.2% | 11.7% |

B=128

Task | FC (oracle) | KiaOmni-G * | KiaOmni- σ 8 * | BlockSal | Ada-SnapKV | H2O | RealSnapKV |

| :—||:—:|:—:|:—:|:—:|:—:|:—:|:—:| | 2wikimqa | 91.1% | **80.0%** | 71.1% | 62.2% | 68.9% | 62.2% | 60.0% | | gov_report | 8.9% | 2.2% | 2.2% | 0.0% | **6.7%** | 0.0% | 0.0% | | hotpotqa | 80.0% | 37.8% | **53.3%** | 35.6% | 44.4% | 40.0% | 20.0% | | multifieldda_en | 82.2% | 62.2% | 60.0% | **64.4%** | 28.9% | 26.7% | 6.7% | | musique | 15.6% | **11.1%** | 6.7% | 8.9% | 4.4% | 4.4% | 0.0% | | narrativeqa | 33.3% | 4.4% | 13.3% | **17.8%** | 8.9% | 11.1% | 2.2% | | qasper | 62.2% | 22.2% | 24.4% | **26.7%** | 22.2% | 20.0% | 15.6% | | qmsum | 8.9% | 0.0% | 0.0% | 0.0% | 0.0% | 0.0% | | **Macro Avg** | 47.8% | 27.5% | **28.9%** | 27.0% | 23.1% | 20.6% | 13.1% |

B=256

Task | FC (oracle) | KiaOmni-G * | KiaOmni- σ 8 * | BlockSal | Ada-SnapKV | H2O | RealSnapKV |

| :—||:—:|:—:|:—:|:—:|:—:|:—:|:—:| | 2wikimqa | 91.1% | 77.8% | **82.2%** | 80.0% | 66.7% | 68.9% | 66.7% | | gov_report | 6.7% | 6.7% | 11.1% | **13.3%** | 11.1% | 6.7% | 0.0% | | hotpotqa | 80.0% | **66.7%** | 60.0% | 60.0% | 60.0% | 55.6% | 51.1% | | multifieldda_en | 82.2% | 73.3% | **84.4%** | 82.2% | 51.1% | 26.7% | 28.9% | | musique | 15.6% | 11.1% | **13.3%** | 11.1% | 11.1% | 8.9% | 2.2% | | narrativeqa | 33.3% | 8.9% | 13.3% | 15.6% | **17.8%** | 11.1% | 6.7% | | qasper | 57.8% | 28.9% | **35.6%** | 33.3% | 24.4% | 20.0% | 22.2% | | qmsum | 6.7% | 0.0% | 0.0% | 0.0% | 0.0% | **2.2%** | 0.0% | | **Macro Avg** | 46.7% | 34.2% | **37.5%** | 36.9% | 30.3% | 25.0% | 22.2% |

B=512

Task | FC (oracle) | KiaOmni-G * | KiaOmni- σ 8 * | BlockSal | Ada-SnapKV | H2O | RealSnapKV |

| :—||:—:|:—:|:—:|:—:|:—:|:—:|:—:| | 2wikimqa | 91.1% | 84.4% | **88.9%** | 82.2% | 86.7% | 75.6% | 66.7% | | gov_report | 8.9% | 8.9% | **15.6%** | 8.9% | 11.1% | 8.9% | 6.7% | | hotpotqa | 80.0% | 73.3% | **75.6%** | 66.7% | 68.9% | 57.8% | 68.9% | | multifieldda_en | 82.2% | **82.2%** | 80.0% | 80.0% | 62.2% | 62.2% | 60.0% | | musique | 15.6% | **13.3%** | 11.1% | 11.1% | 8.9% | 6.7% | 11.1% | | narrativeqa | 33.3% | 24.4% | **26.7%** | **26.7%** | 15.6% | 20.0% | 8.9% | | qasper | 60.0% | **46.7%** | 31.1% | 42.2% | 31.1% | 22.2% | 17.8% | | qmsum | 8.9% | **6.7%** | 2.2% | 0.0% | 4.4% | 0.0% | 2.2% | | **Macro Avg** | 47.5% | **42.5%** | 41.4% | 39.7% | 36.1% | 31.7% | 30.3% |

0.16.2 Mistral-7B**B=98**

Task | FC (oracle) | KiaOmni-G * | KiaOmni- σ 8 * | BlockSal | Ada-SnapKV | H2O | RealSnapKV |

| :—||:—:|:—:|:—:|:—:|:—:|:—:|:—:| | 2wikimqa | 84.4% | **57.8%** | 46.7% | 53.3% | 55.6% | 53.3% | 53.3% | | gov_report | 2.2% | 0.0% | **2.2%** | 0.0% | 0.0% | 0.0% | 0.0% | | hotpotqa | 68.9% | 46.7% | 51.1% | **62.2%** | 46.7% | 55.6% | 33.3% | | multifieldda_en | 84.4% | 22.2% | 22.2% | 22.2% | **26.7%** | 24.4% | 13.3% | | musique | 20.0% | 6.7% | 8.9% | **13.3%** | 2.2% | 2.2% | 6.7% | | narrativeqa | 42.2% | 11.1% | 13.3% | 11.1% | 17.8% | **20.0%** | **20.0%** | | qasper | 55.6% | **28.9%** | 22.2% | 20.0% | 17.8% | 13.3% | 20.0% | | qmsum | 8.9% | 0.0% | 0.0% | 0.0% | 0.0% | 0.0% | 0.0% | | **Macro Avg** | 45.8% | 21.7% | 20.8% | **22.8%** | 20.9% | 21.1% | 18.3% |

B=256

Task	FC (oracle)	KiaOmni-G *	KiaOmni- σ 8 *	BlockSal	Ada-SnapKV	H2O	RealSnapKV
bio_niah_gene	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
bio_niah_single	20.0%	0.0%	0.0%	13.3%	0.0%	0.0%	0.0%
bio_vt	6.7%	6.7%	6.7%	20.0%	0.0%	0.0%	0.0%
clinical_niah	100.0%	100.0%	100.0%	100.0%	100.0%	66.7%	46.7%
medalpaca_medqa	60.0%	46.7%	53.3%	53.3%	53.3%	46.7%	86.7%
medmcqa	86.7%	73.3%	80.0%	73.3%	86.7%	80.0%	73.3%
pubmedqa	66.7%	60.0%	60.0%	60.0%	66.7%	53.3%	46.7%
Macro Avg	48.6%	41.0%	42.9%	45.7%	43.8%	40.9%	33.3%

B=512

Task	FC (oracle)	KiaOmni-G *	KiaOmni- σ 8 *	BlockSal	Ada-SnapKV	H2O	RealSnapKV
bio_niah_gene	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
bio_niah_single	20.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
bio_vt	6.7%	10.0%	0.0%	10.0%	10.0%	3.3%	0.0%
clinical_niah	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%
medalpaca_medqa	60.0%	60.0%	60.0%	60.0%	60.0%	60.0%	60.0%
medmcqa	86.7%	86.7%	86.7%	80.0%	86.7%	86.7%	86.7%
pubmedqa	66.7%	66.7%	66.7%	66.7%	66.7%	66.7%	66.7%
Macro Avg	48.6%	46.2%	44.8%	46.2%	46.2%	45.2%	42.9%

0.17 References

1. Zhang, Z., Sheng, Y., Zhou, T., et al. *H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models*. NeurIPS 2023. arXiv:2306.14048.
2. Li, Y., Huang, Y., Yang, B., et al. *SnapKV: LLM Knows What You are Looking for Before Generation*. NeurIPS 2024. arXiv:2404.14469.
3. Hsieh, C.-P., Sun, S., Krizan, S., et al. *RULER: What’s the Real Context Size of Your Long-Context Language Models?* arXiv:2404.06654, 2024.
4. Bai, Y., Lv, X., Zhang, J., et al. *LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding*. ACL 2024. arXiv:2308.14508.
5. Zheng, L., Chiang, W.-L., Sheng, Y., et al. *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. NeurIPS 2023. arXiv:2306.05685.
6. Qwen Team. *Qwen2.5 Technical Report*. arXiv:2412.15115, 2025.
7. Jiang, A. Q., Sablayrolles, A., Mensch, A., et al. *Mistral 7B*. arXiv:2310.06825, 2023.
8. Almazrouei, E., et al. *The Falcon Series of Open Language Models*. arXiv:2311.16867, 2023.
9. Labrak, Y., Bazoge, A., Morin, E., et al. *BioMistral: A Collection of Open-Source Pre-trained Large Language Models for Medical Domains*. arXiv:2402.10373, 2024.
10. Megiddo, N. & Modha, D. S. *ARC: A Self-Tuning, Low Overhead Replacement Cache*. FAST 2003.
11. Cai, Z., Zhang, Y., Gao, B., et al. *PyramidKV: Dynamic KV Cache Compression based on Pyramidal Information Funneling*. arXiv:2406.02069, 2024.
12. Sheng, Y., Zheng, L., Yuan, B., et al. *FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU*. ICML 2023. arXiv:2303.06865.

Revised Draft — Aliwey Abood — 2026-06-10
Experiments 001–039 · Platforms: Kaggle T4×2, Modal A10G/A100/L4, Lightning AI L4
Total judged samples: 61,681 · Architectures tested: 4